MSA-DATB09-03 · NEURAL NETWORKS & DEEP LEARNING FOUNDATIONS

Session 5 / 8

Vision project: activations and initialisation

You classify clothes with your network, make it deeper, watch it fail, and fix it: ReLU, Xavier and He.

Guillaume Saint-Cirgue Albert School · MSc 1 October 2026

Guillaume Saint-Cirgue your instructor

Data Scientist and ML Engineer since 2016

2013 to 2016	Engineering degree, **ENSMM** (Besançon)
2016 to 2017	Data Scientist, **Rolls-Royce**
2017 to 2019	Lead ML Engineer, **GKN Aerospace**
2019	Launched **Machine Learnia** on YouTube: 200,000+ subscribers today
2020	Founded my company and my startup: **30+ AI projects** delivered since
2023	Head of Data, **Airbus**
Today	**ML-PRO** training program: 800 students

To go further

MY YOUTUBE CHANNEL

Machine Learnia

200,000+ subscribers. Machine learning and deep learning videos, including a neural network coded from scratch in NumPy.

youtube.com/@MachineLearnia

MY EBOOK

Apprendre le Machine Learning en une semaine

In English: Learn Machine Learning in One Week.

machinelearnia.com/apprendre-le-machine-learning-en-une-semaine

Both resources are in French. Click the cards in your copy of the course to open them.

Today's project: recognise clothes



FashionMNIST (Zalando, 2017): two training images per class.

- 70 000 grey images of 28×28 pixels, 10 classes.
- 60 000 for training (we keep 10 000 of them for validation), 10 000 for the test.
- Harder than the 8×8 digits of Session 4: shirt, T-shirt, pullover and coat look alike.
- Your tool: the network and the training loop of Session 4.

From an image to a vector

A dense layer takes a vector: we **flatten** the 28×28 grid into $x \in \mathbb{R}^{784}$, row after row.

$$\underbrace{(B, 28, 28)}_{\text{batch of images}} \xrightarrow{\texttt{nn.Flatten()}} \underbrace{(B, 784)}_{X} \xrightarrow{\text{784-128-10}} \underbrace{(B, 10)}_{\text{logits}}$$

- Standardise the pixels: $x \leftarrow (x - \mu)/s$, with μ, s computed on the **training** images only.
- The network of Session 4: one hidden layer of 128 sigmoid neurons.

$$\underbrace{784 \times 128 + 128}_{100\,480} + \underbrace{128 \times 10 + 10}_{1\,290} = 101\,770 \text{ parameters}$$

```
model = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 128),
    nn.Sigmoid(),
    nn.Linear(128, 10),
)
```

Flattening throws away the 2-D structure: pixel 0 and pixel 28 are neighbours in the image, but just two coordinates for the network. We fix that in Session 7 (convolutions).

The recipe: everything from Session 4

```
train_set = datasets.FashionMNIST(root=DATA_DIR, train=True, download=True)
X_full = train_set.data.float() / 255            # (60000, 28, 28), pixels in [0, 1]
y_full = train_set.targets                       # (60000,), int64
# split train / validation, standardise with the training mean and std, then:
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=128, shuffle=True)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.2)
history = fit(model, train_loader, val_loader, loss_fn, optimizer, n_epochs=10)
```

- `.data` and `.targets` give all the images at once as tensors: much faster on a CPU than reading them one by one.
- `fit`, `train_one_epoch`, `evaluate`: your functions of TP04, given at the top of the notebook.

TP · Part A: your baseline

TP 40 min tp/TP05_vision_project.ipynb, Part A

1. Load FashionMNIST, split, standardise, build the three DataLoaders.
2. Build the 784-128-10 sigmoid network, train it 10 epochs (SGD, lr 0.2).
3. Measure the test accuracy once; compute the confusion matrix; find the most frequent mistake.
4. Look at the misclassified images: would you have done better?

Deliverable: Exercises 1 to 8 validated, your test accuracy and your most frequent mistake.

Your notes / answers (saved in this browser)

Make it deeper?

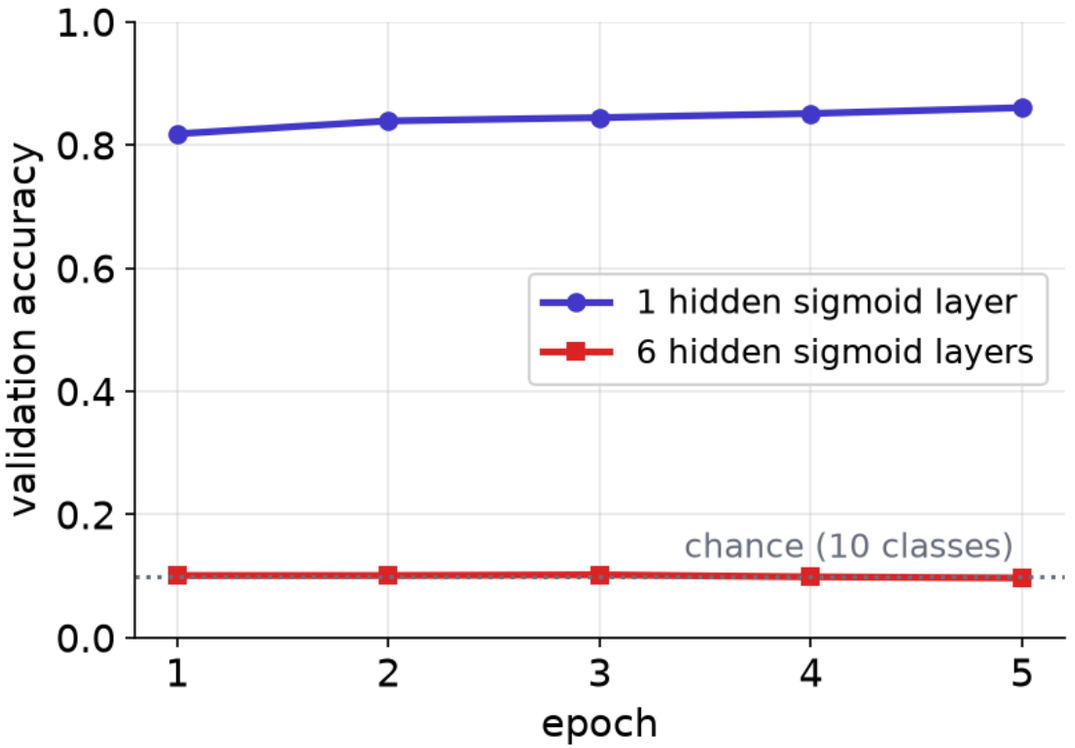
More layers means more combinations of features. Same data, same SGD, same learning rate, but 6 hidden layers of 128 sigmoid neurons instead of 1.

Predict 2 min vote

1. After 5 epochs, the 6-layer network reaches: (a) more than the 1-layer network, (b) about the same, (c) less, (d) about 10 %.
2. Justify in one sentence.

Your notes / answers (saved in this browser)

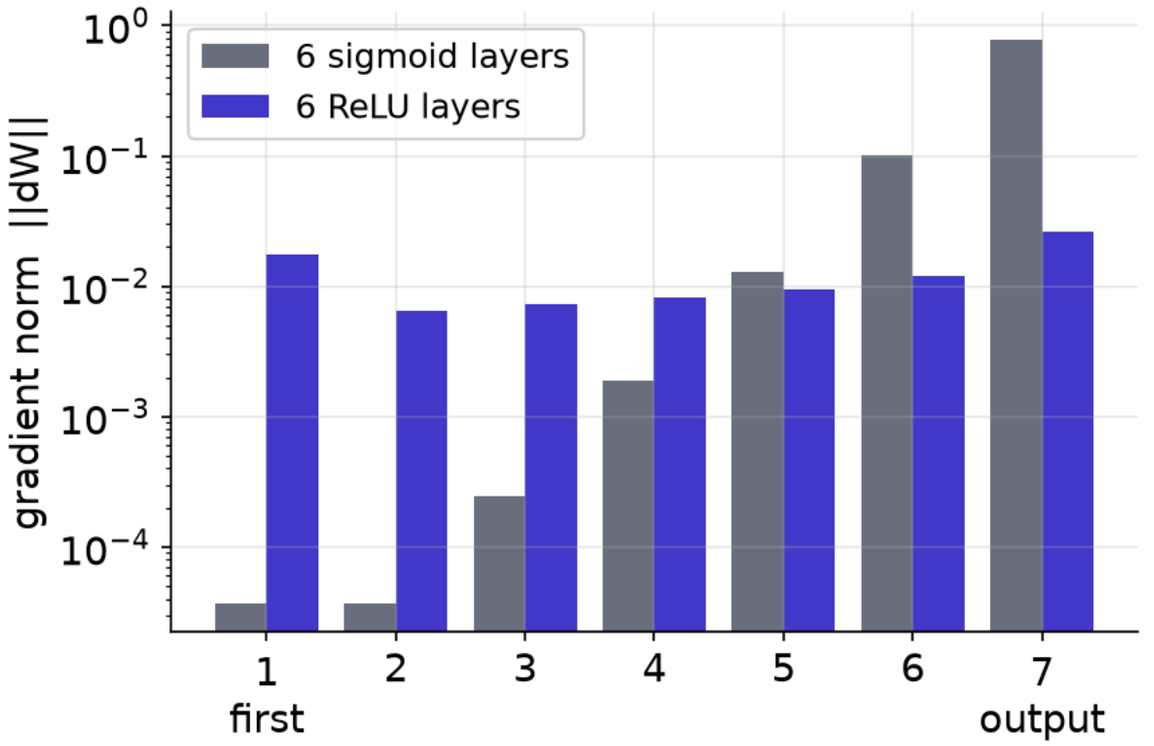
Deeper is worse: the network does not learn at all



FashionMNIST, SGD lr 0.2, batch 128, 50 000 training images.

- 1 hidden layer: 82 % after one epoch, 86 % after five.
- 6 hidden layers: 10 %, chance level. The loss stays at $\ln 10 = 2.30$.
- Answer: (d). More parameters, not a bug in the code. So what is going on?

Where does learning stop? The gradient of each layer



Norm of $dW^{[l]}$ after one backward pass on 256 images, at initialisation.

- Sigmoid (grey): going backwards, the gradient is divided by about 7 at almost every layer. First layer 3.6×10^{-5}, output layer 0.77: **20 000 times less**.
- The first layers barely move: their outputs stay random, the last layers have nothing useful to learn from.
- ReLU (blue): all layers of the same order. Why?

Backpropagation multiplies one derivative per layer

Take a chain of one neuron per layer: $z^{[l]} = w^{[l]} a^{[l-1]} + b^{[l]}$, $a^{[l]} = \sigma(z^{[l]})$. The chain rule (Session 3) gives

$$\frac{\partial a^{[L]}}{\partial a^{[0]}} = \prod_{l=1}^{L} \frac{\partial a^{[l]}}{\partial a^{[l-1]}} = \prod_{l=1}^{L} \underbrace{\frac{\partial a^{[l]}}{\partial z^{[l]}}}_{\sigma'(z^{[l]})} \underbrace{\frac{\partial z^{[l]}}{\partial a^{[l-1]}}}_{w^{[l]}} = \prod_{l=1}^{L} w^{[l]}\, \sigma'(z^{[l]})$$

With matrices it is the same structure, $dZ^{[l]} = W^{[l+1]\top} dZ^{[l+1]} \odot \sigma'(Z^{[l]})$: every layer we go back multiplies the gradient by the weights and by σ'.

Consequence

The gradient reaching the first layer is a product of L factors. If each factor is below 1, it shrinks **exponentially** with the depth: the vanishing gradient. If each is above 1, it explodes.

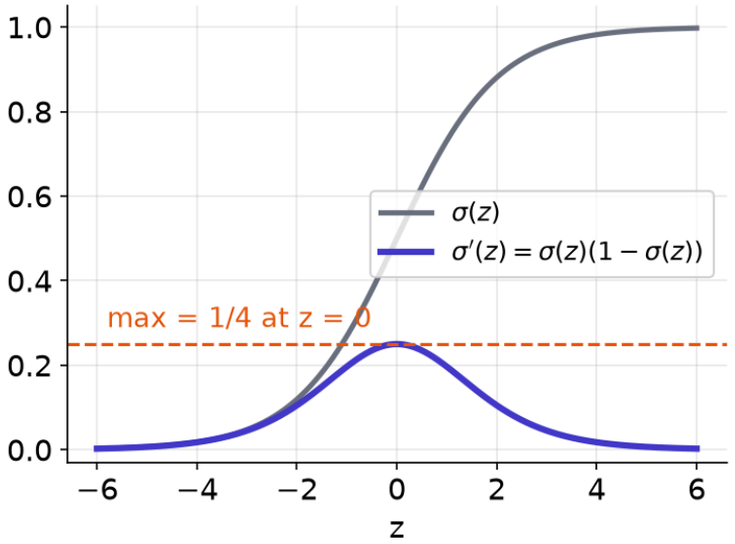
The derivative of the sigmoid is at most 1/4

From Session 2: $\sigma'(z) = \sigma(z)\,(1 - \sigma(z))$. Write $s = \sigma(z) \in (0, 1)$:

$$g(s) = s(1 - s), \qquad g'(s) = 1 - 2s = 0 \iff s = \tfrac{1}{2}$$

$$\max_z \sigma'(z) = \tfrac{1}{2}\left(1 - \tfrac{1}{2}\right) = \tfrac{1}{4}, \quad \text{reached at } z = 0$$

- Even in the best case, each sigmoid layer divides the gradient by 4.
- When $|z|$ is large the neuron **saturates**: $\sigma'(5) = 0.0066$.



The sigmoid and its derivative.

Exercise 1 · How fast does the gradient vanish?

A chain of sigmoid neurons, one per layer, all weights $w^{[l]} = 1$.

Exercise 7 min pen and paper

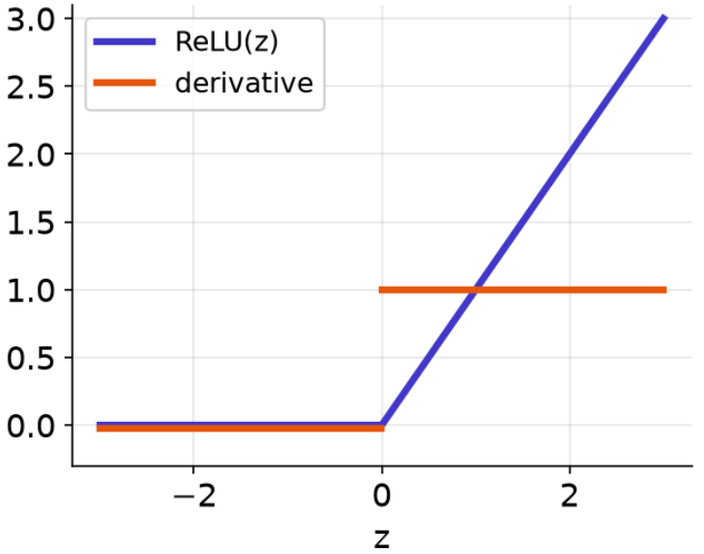
1. Give an upper bound of $|\partial a^{[5]} / \partial a^{[0]}|$ for 5 layers, then for 10 layers.
2. In practice the $z^{[l]}$ are not all 0. With $z = (0.5, -1, 2, 1, -0.5)$ and $\sigma(z) = (0.622, 0.269, 0.881, 0.731, 0.378)$, compute the five $\sigma'(z^{[l]})$ and their product.
3. The last layer receives a gradient of order 1 and $\eta = 0.1$. What is the order of the update of the first layer's weight with 5 layers?
4. Could we fix it with large weights, say $w = 4$, so that $w\,\sigma'(0) = 1$?

Your notes / answers (saved in this browser)

The fix: ReLU

$$\mathrm{ReLU}(z) = \max(0, z), \qquad \mathrm{ReLU}'(z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$$

- On an active path ($z > 0$) the derivative is exactly 1: the product of derivatives does not shrink with depth.
- Does not saturate for $z > 0$.
- Very cheap: a comparison, no exponential.
- Still non-linear (the kink at 0), so stacking layers is useful (Session 3).
- Popularised by Nair and Hinton (2010), Glorot et al. (2011), then AlexNet (2012): the default for hidden layers since.



ReLU and its derivative (0 or 1).

Exercise 2 · A ReLU chain, forward and backward

$$
\begin{aligned}
z_1 &= w_1 x + b_1, & a_1 &= \mathrm{ReLU}(z_1) \\
z_2 &= w_2 a_1 + b_2, & a_2 &= \mathrm{ReLU}(z_2) \\
\hat{y} &= w_3 a_2, & \mathcal{L} &= \tfrac{1}{2}(\hat{y} - y)^2
\end{aligned}
$$

$x = 2$, $y = 3$, $w_1 = 0.5$, $b_1 = 1$, $w_2 = -1$, $b_2 = 3$, $w_3 = 2$.

Exercise 7 min pen and paper

1. Forward: compute $z_1, a_1, z_2, a_2, \hat{y}$ and $\mathcal{L}$.
2. Backward: compute $\partial\mathcal{L}/\partial w_3$, $\partial\mathcal{L}/\partial w_2$ and $\partial\mathcal{L}/\partial w_1$.
3. Same question with $b_2 = 0$. What happens to the gradients of w_1 and w_2?

Hint: start from $\partial\mathcal{L}/\partial\hat{y} = \hat{y} - y$ and multiply by one local derivative per step.

Your notes / answers (saved in this browser)

The price of ReLU: dead neurons

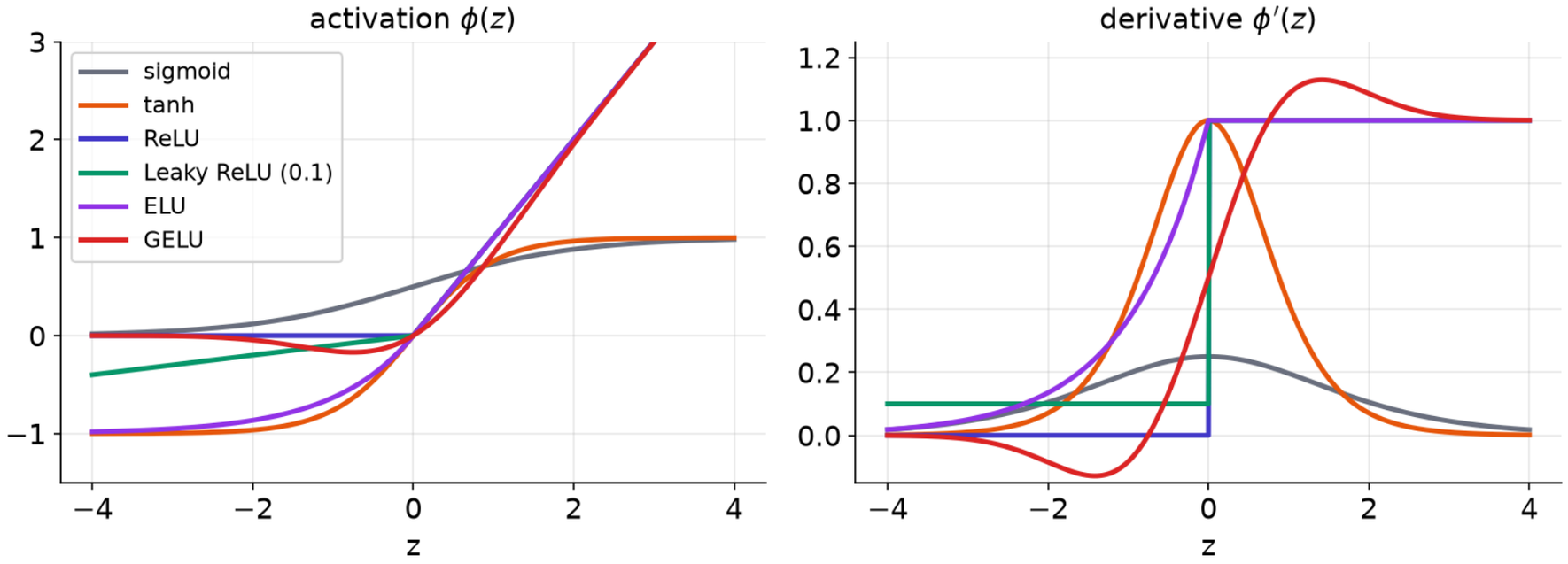
- If $z = w^\top x + b < 0$ for **every** input, the neuron outputs 0 and its derivative is 0 everywhere.
- No gradient reaches w and b: it never comes back. A **dead ReLU**.
- Typical causes: a learning rate too large (one big step pushes b far below 0), a bad initialisation.

Fixes

- A reasonable learning rate.
- A good initialisation (He, in a few slides).
- Leaky ReLU: $\max(0.01z, z)$, a small slope for $z < 0$, so the gradient is never exactly 0.

TP Exercise 16: you set the biases of a layer to -10 and observe 100 % dead units and a gradient of exactly 0.

The other activation functions



$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} = 2\sigma(2z) - 1$: centred on 0, $\tanh'(0) = 1$, still saturates.

Leaky ReLU: $\max(\alpha z, z)$, $\alpha = 0.01$. ELU: z if $z > 0$, $e^z - 1$ otherwise (smooth).

GELU: $z\,\Phi(z)$, Φ the standard normal CDF; a smooth ReLU, used in transformers (BERT, GPT).

Which activation where?

Activation	Where	Why
ReLU	hidden layers, default choice (MLP, CNN)	derivative 1, cheap, works
Leaky ReLU, ELU	hidden layers, if many units die	never exactly zero gradient
GELU	hidden layers of transformers	smooth, slightly better in large models
tanh	inside RNN and LSTM cells (Session 8)	bounded in $(-1, 1)$, centred on 0
sigmoid	output for 2 classes or multi-label; gates of LSTM	a probability in $(0, 1)$; avoid in deep hidden stacks
softmax	output for C exclusive classes only	C probabilities summing to 1 (in PyTorch, inside `CrossEntropyLoss`)

Output layer: chosen by the task (probability, class distribution, or nothing for regression). Hidden layers: chosen for the gradient flow.

Initialisation: why not all zeros?

Suppose all the weights of a layer start equal (zeros, or any constant c).

1. **Forward**: every neuron j of the layer computes $z_j = \sum_i c\, x_i + b$: the same value, so the same activation.
2. **Backward**: $dZ^{[l]} = W^{[l+1]\top} dZ^{[l+1]} \odot \phi'(Z^{[l]})$ has identical rows, so every neuron receives the **same gradient**.
3. **Update**: same value minus same step: they are still equal. By induction, forever.

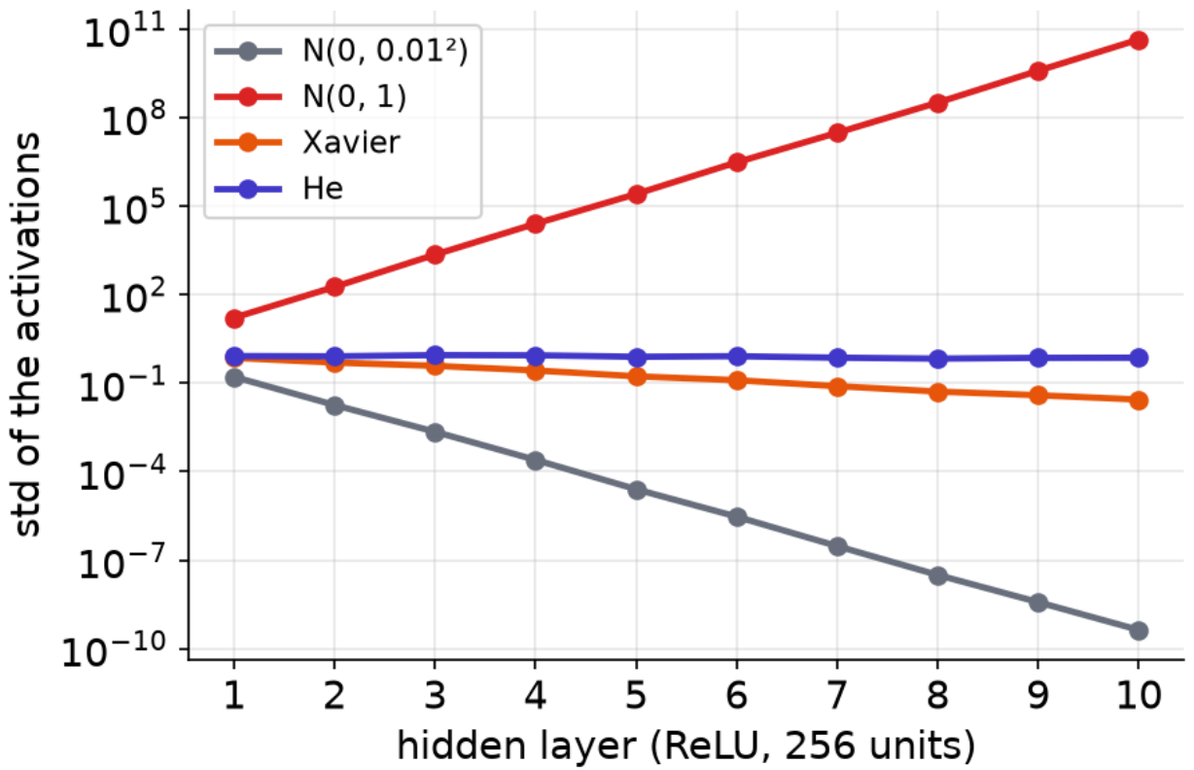
128 identical neurons are worth one neuron. With exactly zero weights, worse: the gradient of the earlier layers is zero.

Break the symmetry

Initialise the weights **randomly**, centred on 0. Biases can start at 0.

But how large should the random values be? Too small or too large is fatal in a deep network.

Too small, too large: the signal through 10 layers



10 hidden ReLU layers of 256 units, 1000 FashionMNIST images, before training.

- $\mathcal{N}(0, 0.01^2)$: each layer multiplies the signal by about 0.11. After 10 layers: 4×10^{-10}. The gradients vanish the same way.
- $\mathcal{N}(0, 1)$: about 11 per layer. After 10 layers: 4×10^{10}. Explodes.
- Xavier: slow decay with ReLU (designed for tanh).
- He: constant. What is the right scale, and why these two?

The variance argument

One neuron: $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$. Assume the w_i and x_i independent, w_i of mean 0, variance $\mathrm{Var}(w)$, and x_i of mean 0, variance $\mathrm{Var}(x)$:

$$\mathrm{Var}(z) = \sum_{i=1}^{n_{\text{in}}} \mathrm{Var}(w_i x_i) = \sum_{i=1}^{n_{\text{in}}} \underbrace{\mathbb{E}[w_i^2]\,\mathbb{E}[x_i^2]}_{\text{means are } 0} = n_{\text{in}}\,\mathrm{Var}(w)\,\mathrm{Var}(x)$$

To keep $\mathrm{Var}(z) = \mathrm{Var}(x)$ layer after layer:

$$n_{\text{in}}\,\mathrm{Var}(w) = 1 \iff \mathrm{Var}(w) = \frac{1}{n_{\text{in}}}$$

Check with the figure ($n_{\text{in}} = 256$, ReLU halves the variance): $\mathcal{N}(0, 0.01^2)$ gives a factor $\sqrt{256 \times 0.01^2/2} = 0.11$ per layer, $\mathcal{N}(0,1)$ gives $\sqrt{256/2} = 11.3$.

The same argument on the backward pass gives $n_{\text{out}}\,\mathrm{Var}(w) = 1$.

Xavier and He initialisation

Xavier / Glorot (2010): tanh, sigmoid

$$\mathrm{Var}(w) = \frac{1}{n_{\text{in}}} \quad \text{or} \quad \mathrm{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

Forward ($1/n_{\text{in}}$) and backward ($1/n_{\text{out}}$) cannot both hold: Glorot takes the harmonic mean.

He / Kaiming (2015): ReLU

$$\mathrm{Var}(w) = \frac{2}{n_{\text{in}}}$$

ReLU zeroes half of the z: $\mathbb{E}[a^2] = \frac{1}{2}\mathrm{Var}(z)$. The factor 2 compensates exactly.

```
layer = nn.Linear(784, 256)
nn.init.xavier_uniform_(layer.weight)
nn.init.kaiming_normal_(layer.weight,
                        nonlinearity="relu")
nn.init.zeros_(layer.bias)
print(layer.weight.std())  # about 0.0505
```

- The trailing `_`: the tensor is filled in place.
- A uniform $U(-a, a)$ has variance $a^2/3$: `xavier_uniform_` uses $a = \sqrt{6/(n_{\text{in}} + n_{\text{out}})}$.
- PyTorch's default for `nn.Linear` is a Kaiming-uniform variant with $a = 1/\sqrt{n_{\text{in}}}$, so $\mathrm{Var}(w) = 1/(3n_{\text{in}})$: the right order, smaller than He.

Exercise 3 · The initial weights of a 784-256 layer

Exercise 5 min pen and paper

1. Give the standard deviation of the initial weights of `nn.Linear(784, 256)` with Xavier $2/(n_{\text{in}} + n_{\text{out}})$, with Xavier $1/n_{\text{in}}$, and with He.
2. Same layer initialised with $\mathcal{N}(0, 1)$, inputs standardised (variance 1). What is the standard deviation of z? What does the sigmoid do with such values?
3. Which initialisation for the 6-layer ReLU network of the TP? For a tanh network?

Hint: $\sqrt{0.001923} \approx 0.0439$, $\sqrt{0.001276} \approx 0.0357$, $\sqrt{0.002551} \approx 0.0505$.

Your notes / answers (saved in this browser)

Break: 10 minutes

TP · Parts B to D: break it, then fix it

TP 57 min tp/TP05_vision_project.ipynb, Parts B to D

1. Part B: `make_mlp(depth, activation)`, $\max \sigma' = 1/4$ with autograd, the gradient norm of each layer, train the 6-layer sigmoid network (it fails).
2. Part C: the same with ReLU; sigmoid, tanh, ReLU, Leaky ReLU in a loop (table); provoke dead ReLUs.
3. Part D: `init_weights` (zeros, small, large, Xavier, He), the std of the activations through 10 layers, then train with small versus He.
4. Going further: a 3-layer ReLU network with He initialisation and Adam, above 88 % on the test set.

Deliverable: the activation table of Exercise 15 and the plot of Exercise 18.

Your notes / answers (saved in this browser)

Watch it on YouTube Machine Learnia, Deep Learning series

A simple neuron on images, and the general network with N layers.

Video 6

Application : chiens et chats

Application: cats versus dogs

Video 10

Réseau de neurones à N couches

A neural network with N layers

Playlist

The whole Deep Learning series

10 videos, from the perceptron to the network with N layers

Videos in French. Click a line in your copy of the course to open the video.

Summary

1. A one-hidden-layer network reaches about 85 % on FashionMNIST; with 6 sigmoid layers it does not learn at all.
2. Backprop multiplies one $w\,\phi'(z)$ per layer; $\sigma' \leq 1/4$, so the gradient vanishes exponentially with depth.
3. ReLU (derivative 1 on active paths) is the default for hidden layers; watch for dead units (Leaky ReLU). GELU in transformers, tanh in RNNs, sigmoid and softmax at the output.
4. Initialise randomly to break the symmetry, with the right variance: $\mathrm{Var}(w) = 1/n_{\text{in}}$ or $2/(n_{\text{in}} + n_{\text{out}})$ (Xavier, tanh) and $2/n_{\text{in}}$ (He, ReLU).

Next session: more tools against the same family of problems and against overfitting: batch normalisation, dropout, weight decay, better optimisers (momentum, Adam), and how to diagnose a training from its curves.